1. The Main Question

In the previous class, we created:

datax_db.orders

But one important question remains:

> Where is the actual data stored, and who manages its lifecycle?

This leads to two important table types:

Managed Table
External Table

Both have metadata and actual data in storage, but the **ownership of the data lifecycle** is different.

---

2. Managed Table

A managed table is a table where:

- Databricks/metastore manages the table metadata.
- We do not manually specify an external storage location.
- The platform determines the managed storage location.
- The table's data lifecycle is associated with the table.

Example:

```
orders_df.write \
   .format("delta") \
   .mode("overwrite") \
   .saveAsTable("datax_db.orders")
```

Notice that we did not provide:

LOCATION '/some/path'

Simple Mental Model

Managed Table
   ↓

Metadata
+
Managed Data Lifecycle

Important:

The actual business data still exists in storage.

The metastore stores metadata about the table, not all the actual records.

---

3. Inspecting a Managed Table

Use:

DESCRIBE EXTENDED datax_db.orders;

Important information includes:

- Provider
- Type
- Location

The `Location` tells us where the actual data is physically stored.

The important point is:

> We did not manually choose this location while creating the managed table.

Relationship:

Hive Metastore
↓
Table Metadata

Storage Location
↓
Actual Delta Data

---

4. Finding the Physical Location

You can inspect the table details:

```
managed_detail = spark.sql(
    "DESCRIBE DETAIL datax_db.orders"
)

display(managed_detail)
```

To capture the location:

```
managed_location = (
    spark.sql("DESCRIBE DETAIL datax_db.orders")
    .select("location")
    .first()[0]
)

print(managed_location)
```

The exact physical path can vary depending on the Databricks environment.

---

5. Physical Data Still Exists

Inspect the underlying managed table files:

```
display(dbutils.fs.ls(managed_location))
```

You may see:

```
_delta_log
part-....
```

This proves that the table has actual Delta data stored physically.

Remember

Table Metadata
≠
Actual Data

---

6. What Is an External Table?

An external table has:

- Metadata registered in the metastore
- Data stored at an explicitly specified location
- Storage lifecycle independent from the table metadata

Simple view:

Hive Metastore
↓
Table Metadata
↓
datax_db.orders_external
↓
Explicit Storage Location
↓
Actual Delta Files

The key difference:

> For an external table, we explicitly tell Databricks where the underlying data lives.

---

7. Why External Tables Are Useful

Imagine:

- Another pipeline already writes the data.
- Another application owns the storage.
- Multiple systems depend on the same data.
- The organization wants explicit control over the physical location.

In such cases, dropping the table should not automatically delete the underlying externally owned data.

That is where an external table becomes useful.

---

8. Creating Data at an Explicit Location

First define a storage location:

external_orders_path = "<external-demo-path>/orders"

Write Delta data there:

```
orders_df.write \
    .format("delta") \
    .mode("overwrite") \
    .save(external_orders_path)
```

At this point:

```
Delta Data
  ↓
Physical Location
```

But we still do **not** have a registered table.

Important

Writing Delta files to a location does not automatically create a table.

---

9. Registering the External Table

Now create metadata that points to that existing Delta location.

```
CREATE TABLE datax_db.orders_external
USING DELTA
LOCATION '<external-demo-path>/orders';
```

Breakdown:

CREATE TABLE datax_db.orders_external

Creates/registers the table.

USING DELTA

Specifies the underlying data format.

LOCATION '...'

Specifies exactly where the data exists.

Most Important Keyword

LOCATION

This is what makes the storage location explicitly controlled.

---

10. Querying the External Table

```
SELECT *
FROM datax_db.orders_external;
```

From a user's perspective:

SELECT * FROM datax_db.orders;

and:

SELECT * FROM datax_db.orders_external;

both behave like tables.

But their **storage lifecycle ownership** is different.

---

11. Managed vs External Table

Concept	Managed Table	External Table
Metadata	Metastore	Metastore
Actual data	Storage	Storage
Location explicitly specified by us	Usually No	Yes
Data lifecycle tied to table	Yes	No
`DROP TABLE` removes metadata	Yes	Yes
Underlying table data after drop	Managed lifecycle removed	External data remains

Key Idea

The word **external** does not mean Databricks cannot query the data.

It means the underlying data lifecycle exists independently of the table definition.

---

12. Most Important Practical: DROP TABLE

This is the most important concept from this class.

External Table DROP

Suppose:

DROP TABLE datax_db.orders_external;

The table disappears from the metastore.

But the external Delta files remain.

DROP EXTERNAL TABLE
↓
Metadata Removed
↓
Underlying Data Remains

Verify:

display(dbutils.fs.ls(external_orders_path))

The files should still exist.

---

13. External Data Can Still Be Read

Even after the external table is dropped:

```
check_df = (
    spark.read
    .format("delta")
    .load(external_orders_path)
)

display(check_df)
```

The data is still available.

Important Line

The table disappeared.
The data did not.

---

14. External Table Can Be Recreated

Because the external files still exist, the table can be registered again.

```
CREATE TABLE datax_db.orders_external
USING DELTA
LOCATION '<external-demo-path>/orders';
```

Then:

```
SELECT *
FROM datax_db.orders_external;
```

The same data becomes queryable again.

This demonstrates:

Metadata ≠ Data

We recreated metadata pointing to existing data.

---

15. Managed Table DROP

For a managed table:

DROP TABLE datax_db.managed_demo;

Conceptually:

MANAGED TABLE

DROP TABLE
↓
Metadata Removed
+
Managed Data Lifecycle Removed

Whereas:

EXTERNAL TABLE

DROP TABLE
↓
Metadata Removed

BUT

External Data Remains

The main point is the lifecycle relationship, not the exact timing of physical cleanup implementation details.

---

16. `.save()` vs `.saveAsTable()`

This is a useful beginner mental model.

`.save()`

```
orders_df.write \
    .format("delta") \
    .save(path)
```

Think:

LOCATION

It primarily writes data to a physical location.

`.saveAsTable()`

```
orders_df.write \
    .format("delta") \
    .saveAsTable("datax_db.orders")
```

Think:

TABLE

It creates/writes through a named table.

Memory Trick

save()
→ Think PATH / LOCATION

saveAsTable()
→ Think TABLE

---

17. When Managed Tables Make Sense

Managed tables can make sense when:

- Databricks manages the complete table lifecycle.
- You do not require independent control of the physical storage location.
- You want simpler table management.

---

18. When External Tables Make Sense

External tables can make sense when:

- Data already exists at a known location.
- Other systems depend on that storage.
- Storage lifecycle should remain independent.
- The organization needs explicit control over data location.

Important

Neither is automatically better.

The decision depends on:

> Who should own and manage the underlying data lifecycle?

---

19. Common Errors

Location Does Not Contain Delta

Example:

```
CREATE TABLE datax_db.orders_external
USING DELTA
LOCATION '<some-path>';
```

If the path contains non-Delta data, Databricks cannot treat it as an existing Delta dataset.

Check:

- Is the location correct?
- Is the dataset actually Delta?
- Is `_delta_log` present where expected?

Wrong Location

Possible error:

PATH_NOT_FOUND

Check:

```
display(dbutils.fs.ls(external_orders_path))
```

Always verify the storage path before assuming the SQL is wrong.

---

20. Practical Debugging Habit

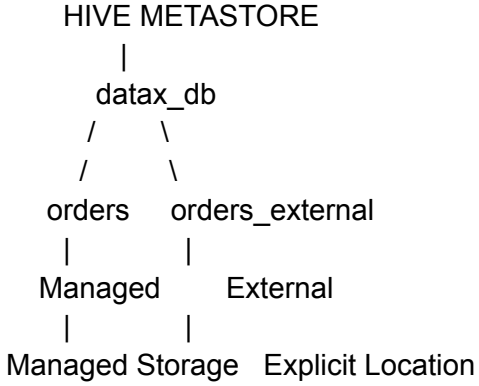
When an external-table operation fails, first ask:

Does the path exist?
↓
Does it contain the expected format?
↓
Is the table pointing to the correct location?

Do not immediately assume the table definition is the issue.

---

21. Complete Conceptual View



Both tables:

- Have metadata
- Have actual data in storage
- Can be queried

The difference is **storage lifecycle ownership**.

---

22. Quick Revision

Managed Table

Metastore

↓
Table Metadata
+
Managed Data Lifecycle

External Table

Metastore
↓
Table Metadata
↓
Points To
↓
Externally Controlled Data

DROP Behavior

DROP Managed Table
→ Metadata removed
→ Managed table lifecycle includes the data

DROP External Table
→ Metadata removed
→ External data remains

---

23. Student Practice

Create product data:

product_id,product_name,category,price
P101,Keyboard,Electronics,899
P102,Mouse,Electronics,499
P103,Monitor,Electronics,14999
P104,Desk,Furniture,7999
P105,Chair,Furniture,5999

Task 1 — Managed Table

Create:

datax_db.products_managed

using:

.saveAsTable()

Task 2 — External Data

Write the same DataFrame to a custom location using:

.save(path)

Task 3 — External Table

Register:

datax_db.products_external

using:

USING DELTA
LOCATION '...'

Task 4 — Compare Locations

Run:

DESCRIBE DETAIL datax_db.products_managed;

and:

DESCRIBE DETAIL datax_db.products_external;

Identify the storage locations.

Task 5 — Test DROP Behavior

Drop:

DROP TABLE datax_db.products_external;

Then check whether the physical Delta data still exists.

Finally, recreate the external table using the same existing files.

---

24. Interview-Relevant Questions

1. What is the difference between a managed table and an external table?

A managed table has its metadata and managed data lifecycle tied together.

An external table has metadata registered in the metastore but points to data at an explicitly controlled external location.

---

2. Does an external table still have metadata?

Yes.

Both managed and external tables have metadata registered in the metastore.

---

3. Where is the actual data stored for both table types?

The actual data exists in storage.

The difference is who controls the storage lifecycle.

---

4. What happens when an external table is dropped?

The table metadata is removed, but the externally stored underlying data remains.

---

5. What happens conceptually when a managed table is dropped?

The table metadata is removed and the platform-managed table data lifecycle is removed with it.

---

6. Why would you use an external table?

When:

- Data already exists
- Multiple systems depend on it
- Storage lifecycle must remain independent
- Explicit control of location is required

---

7. What is the importance of `LOCATION`?

For an external table, `LOCATION` explicitly tells Databricks where the table's underlying data exists.

---

8. Does writing Delta data using `.save(path)` create a table?

No.

It writes Delta data to a path.

A table must still be registered separately if you want to query it using a table name.

---

9. What is the difference between `.save()` and `.saveAsTable()`?

At a beginner level:

.save()
→ Think physical location

.saveAsTable()
→ Think named table

---

10. Can an external table be recreated after being dropped?

Yes, if the underlying external data still exists.

The table metadata can be recreated to point to the same Delta location.

---

11. Is an external table always better than a managed table?

No.

The correct choice depends on who should own and manage the underlying data lifecycle.

Key Takeaway

The most important distinction is not simply where the data is stored.

It is:

Who owns the data lifecycle?

Managed

Table Metadata
+
Managed Data Lifecycle

External

Table Metadata
→ Points to independently controlled data

That difference becomes most visible when the table is dropped.